Hướng dẫn tạo Sequence Check trạng thái

Oct 17, 2026 · @SangNguyen

Tổng quan

Sequence Check mô tả một chuỗi chuyển trạng thái mong đợi của các signal, kèm giới hạn thời gian giữa các bước, để đối chiếu với dữ liệu trong log. Ví dụ: IGN ON → VCU yêu cầu HV → đóng Precharge relay → đóng Main relay → HV ON.

Một sequence gồm bốn phần:

- **Trigger**: điều kiện mở đầu một lần kiểm tra.
- **Steps**: các trạng thái phải xảy ra theo thứ tự, mỗi bước có cửa sổ thời gian và thời gian giữ.
- **Invariants**: điều kiện phải luôn đúng trong suốt lần kiểm tra.
- **Abort**: điều kiện mà khi xảy ra thì lần kiểm tra bị huỷ.

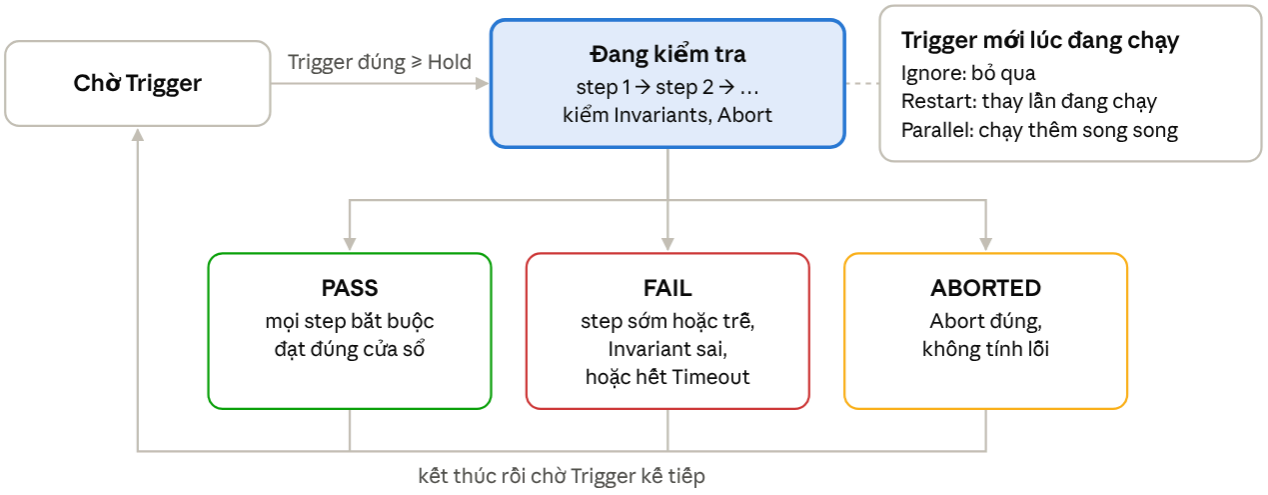
Sequence được tạo trong tab **Configurations** của cửa sổ Graphic và lưu thành file trong thư mục `Sequence Check` của tool.

Cách một lần kiểm tra diễn ra

Mỗi lần Trigger đạt sẽ mở một lần kiểm tra. Lần kiểm tra đi qua các step theo thứ tự và kết thúc bằng một trong các kết quả dưới đây. Bộ kiểm tra tự động sẽ áp dụng đúng các quy tắc này; hiện tại bạn đối chiếu thủ công bằng cột Cursor (xem mục Giới hạn hiện tại).

Kết quả	Khi nào
PASS	Mọi step không Optional đạt trong cửa sổ của nó, và Invariants luôn đúng
FAIL	Một step đạt ngoài cửa sổ (sớm hơn min hoặc muộn hơn max), Invariant bị vi phạm, hoặc hết Timeout khi còn step chưa đạt
ABORTED	Điều kiện Abort đúng trước khi lần kiểm tra kết thúc; không tính là lỗi
Bị thay thế	Chỉ có khi While running = Restart: Trigger mới đến lúc lần này chưa xong

Một lần kiểm tra mở từ Trigger và kết thúc bằng PASS, FAIL hoặc ABORTED



Vòng đời một lần kiểm tra · 3 kết quả, 3 cách xử lý Trigger lặp

Kết thúc theo cách nào thì lần kiểm tra cũng quay về chờ Trigger kế tiếp; Trigger mới đến khi đang kiểm tra được xử lý theo While running.

Quy tắc thời gian

1. **Trigger** được tính khi nhóm điều kiện chuyển từ sai sang đúng (hoặc đã đúng sẵn ở đầu log, nếu chọn) và giữ đúng ít nhất bằng Hold. Mốc t = 0 của lần kiểm tra là lúc điều kiện bắt đầu đúng.
2. **Thời điểm đạt** của một step là lúc điều kiện bắt đầu đúng. Hold chỉ xác nhận và được phép kết thúc sau cửa sổ.
3. **Cửa sổ** [min, max] tính từ mốc After: lúc step trước đạt (step 1 thì là Trigger), lúc Trigger, hoặc lúc một step phía trước đạt. Max = no limit thì chờ tới Timeout.
4. **Optional**: step không đạt cũng không tính lỗi.
5. **Invariants** và **Abort** được kiểm liên tục từ Trigger tới khi lần kiểm tra kết thúc.
6. **Timeout** tính từ Trigger. No limit thì lần kiểm tra kéo dài tới khi xong các step hoặc hết log (Restart: tới Trigger kế tiếp).
7. **While running** quyết định Trigger mới trong lúc đang kiểm tra: Ignore bỏ qua, Restart thay thế lần đang chạy, Parallel mở thêm một lần chạy song song.
8. **Độ phân giải**: signal chỉ cập nhật mỗi chu kỳ message, nên thời điểm đo được có thể trễ tới một chu kỳ. Vì vậy Window nên rộng hơn chu kỳ và Hold nên khoảng 2 chu kỳ.

Câu hỏi mở, sẽ chốt khi làm bộ kiểm tra: nếu điều kiện của step đã đúng sẵn tại mốc After thì tính là đạt tại 0 ms, hay phải chuyển sai rồi đúng lại?

Chuẩn bị

Cần có DBC của các network đã nạp trong CAN Database, vì mọi điều kiện đều chọn signal từ DBC. Không cần kết nối phần cứng.

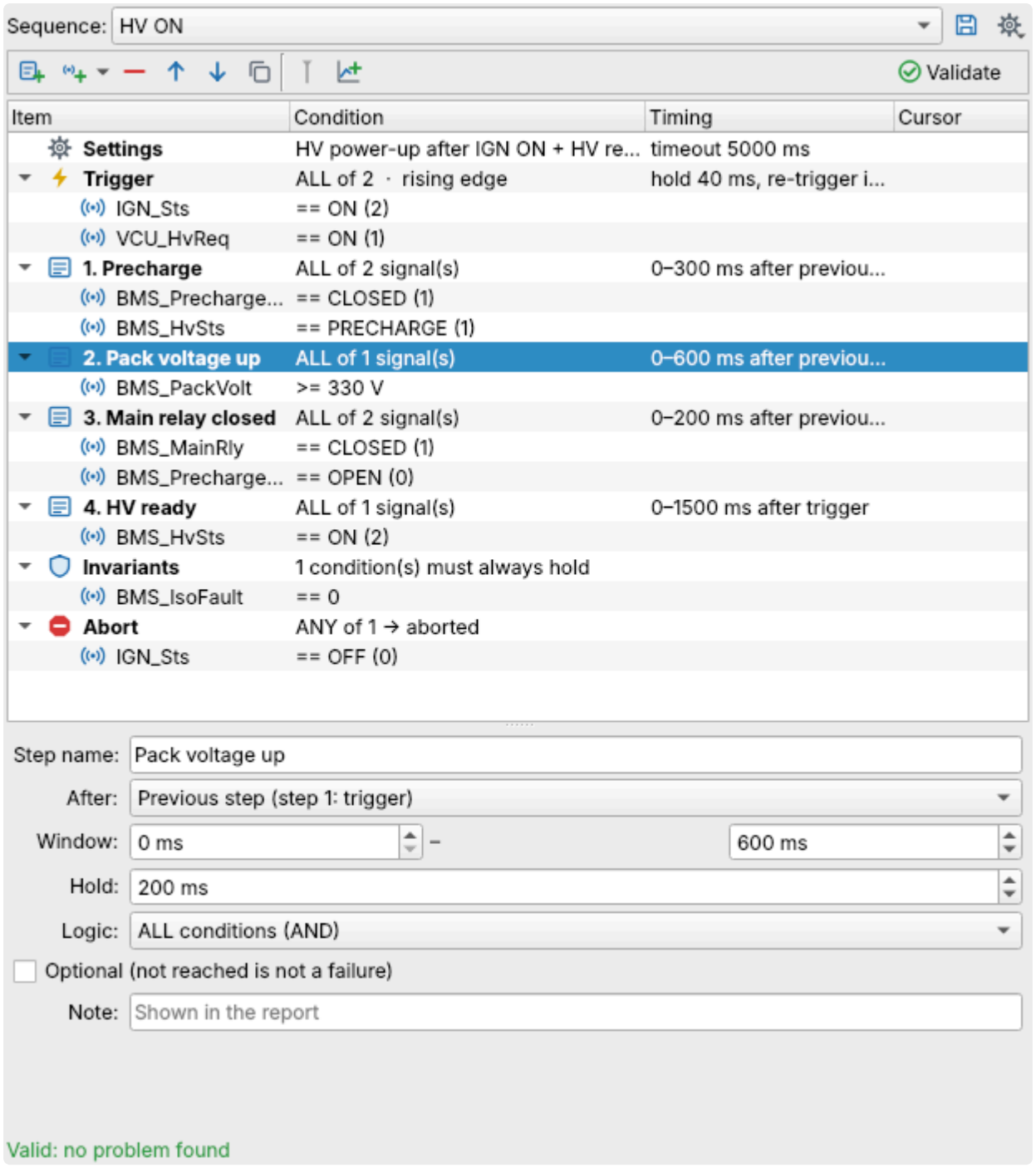
1. Mở **CAN Database** và nạp DBC cho từng network (ví dụ Powertrain, Body_CAN).
2. Mở **Bus › Graphic**.
3. Nếu muốn đối chiếu hoặc lấy giá trị từ log mẫu: chuẩn bị file log (BLF, ASC hoặc TXT của MS_Tool) có một lần chuyển trạng thái đúng.

Nếu chưa nạp DBC, bấm thêm signal sẽ báo "No DBC is loaded".

Màn hình Sequence editor

Trong cửa sổ Graphic, chọn tab dọc **Configurations** bên trái. Màn hình gồm bốn vùng:

- **Dòng Sequence** trên cùng: combo chọn sequence trong thư viện, nút **Save** và nút **...** (New, Save As, Rename, Delete, Import, Export, Open Folder).
- **Thanh công cụ** các nút soạn thảo (bảng dưới).
- **Cây sequence**: Settings, Trigger, các Step, Invariants, Abort. Ba cột sau tên là điều kiện, thời gian và **Cursor** (giá trị tại cursor 1).
- **Bảng thuộc tính** phía dưới: đổi theo dòng đang chọn trên cây.



Ở trên là cây sequence HV ON; phía dưới là bảng thuộc tính của step 2 đang chọn. Dòng cuối báo kết quả Validate.

Nút	Tác dụng
Step	Thêm step ngay sau step đang chọn
Signal	Thêm signal làm điều kiện. Mũi tên bên cạnh: From database... hoặc Signals selected in the Signals tab
Remove	Xoá step hoặc điều kiện đang chọn (phím Del)
Up / Down	Đổi thứ tự step hoặc điều kiện

Nút	Tác dụng
Duplicate	Nhân bản step đang chọn
From cursor	Đặt giá trị mong đợi bằng giá trị của signal tại cursor 1
Plot	Đưa các signal của sequence sang tab Signals để vẽ
Validate	Kiểm tra sequence với DBC đang nạp, hiện danh sách lỗi và cảnh báo

Click phải trên cây cũng có các lệnh thêm step và thêm signal.

Bước 1: Đặt tên và Trigger

Mỗi lần Trigger chuyển từ sai sang đúng là mở đầu một lần kiểm tra mới, nên chọn Trigger là sự kiện khởi đầu chuỗi (ví dụ IGN chuyển sang ON).

1. Bấm **…** › **New**. Sequence mới có tên "New sequence" và có sẵn một step trống "Step 1".
2. Chọn dòng **Settings** và điền:
   - **Name**: tên sequence, cũng là tên file trong thư viện.
   - **Description**: sequence này kiểm tra gì.
   - **Timeout**: thời gian dài nhất của một lần kiểm tra, tính từ Trigger. Để "no limit" thì lần kiểm tra kéo dài tới khi xong các step hoặc hết log (riêng chế độ Restart: tới Trigger kế tiếp).
3. Chọn dòng **Trigger**, bấm **Signal** và chọn signal trong DBC.
4. Chọn **Logic**: "ALL conditions (AND)" hoặc "ANY condition (OR)".
5. Chọn **Start**:
   - "When it becomes true": chỉ tính khi điều kiện chuyển từ sai sang đúng.
   - "Also if already true at the log start": nếu log bắt đầu khi điều kiện đã đúng thì cũng tính là một lần kiểm tra.
6. Đặt **Hold**: Trigger phải đúng liên tục bao lâu mới tính, để một frame nhiễu không tạo ra lần kiểm tra giả. Nên đặt khoảng 2 chu kỳ message của Trigger. Lần kiểm tra vẫn tính từ lúc điều kiện bắt đầu đúng, không phải lúc hết Hold.
7. Chọn **While running**: Trigger mới đến khi một lần kiểm tra đang chạy thì xử lý thế nào.

While running	Trigger mới khi đang kiểm tra
Ignore new triggers (mặc định)	Bỏ qua cho tới khi lần đang chạy kết thúc: PASS, FAIL, ABORTED hoặc hết Timeout
Restart the check	Kết thúc lần đang chạy (ghi là bị thay thế) và bắt đầu lần mới
Start another check in parallel	Mỗi Trigger một lần kiểm tra riêng, nhiều lần chạy cùng lúc

Nếu Trigger còn trống và chưa chọn dòng nào, signal thêm vào sẽ tự vào Trigger.

Sửa một điều kiện

Chọn dòng signal trên cây, rồi chọn **Operator** và **Value** ở bảng thuộc tính. Cách sửa này dùng chung cho Trigger, Step, Invariants và Abort.

Operator	Ý nghĩa	Ví dụ Value
== / !=	Bằng / khác	CLOSED, 2, 0x1F
> / >= / < / <=	So sánh lớn nhỏ	300
between	Trong khoảng a..b, tính cả hai đầu	From 340, To 360
outside	Ngoài khoảng a..b	From 0, To 10
one of	Một trong danh sách	ACC, ON

- **Value** nhận số thập phân, số hex (0x..) hoặc tên trong value table của DBC. Combo Value liệt kê sẵn các tên này.
- Với **one of**, các giá trị cách nhau bằng dấu phẩy.
- **Tolerance** là sai số cho phép của ==, != và one of, dùng cho signal có factor lẻ (ví dụ 0.1).
- Dòng chữ xám dưới ô hiện network, message, ID, unit, range và value table của signal.

Bước 2: Thêm các Step trạng thái

Mỗi Step là một trạng thái phải đạt theo thứ tự: một nhóm điều kiện, cộng với cửa sổ thời gian tính từ một mốc. Step đạt khi nhóm điều kiện trở thành đúng trong cửa sổ và giữ đúng liên tục ít nhất bằng thời gian Hold.

Thời điểm đạt là lúc nhóm điều kiện bắt đầu đúng, không phải lúc hết Hold. Hold chỉ xác nhận và có thể kết thúc sau cửa sổ: với Window 0–300 ms, điều kiện đúng lúc 290 ms và Hold xác nhận xong lúc 340 ms thì step vẫn đạt đúng hạn. Step sau dùng mốc "Previous step" tính từ 290 ms.

1. Step 1 có sẵn khi tạo sequence: chọn nó và đổi tên. Để thêm step khác, chọn step phía trước rồi bấm **Step**: step mới được chèn ngay sau step đang chọn (không chọn step nào thì thêm vào cuối), ô Step name được chọn sẵn để gõ tên.
2. Giữ step đó đang chọn và bấm **Signal** để thêm một hoặc nhiều signal. Muốn lấy đúng các signal đang chọn ở tab Signals thì dùng mũi tên cạnh nút › **Signals selected in the Signals tab**.
3. Chọn từng dòng signal và đặt Operator, Value như Bước 1.
4. Chọn lại dòng step và điền các thuộc tính ở bảng dưới.
5. Lặp lại cho các step sau. Dùng **Up / Down** để đổi thứ tự, **Duplicate** để nhân bản một step gần giống.

Thuộc tính	Ý nghĩa	Mặc định
Step name	Tên hiển thị của step	Step n
After	Mốc tính thời gian: "Previous step (step 1: trigger)", "Trigger" hoặc một step phía trước	Previous step
Window (ms)	Sớm nhất và muộn nhất cho phép sau mốc After. Để mốc trên là "no limit" thì chờ tới Timeout	0 – 1000
Hold (ms)	Điều kiện phải đúng liên tục bao lâu mới tính, để lọc nhiễu. Nên đặt từ 2 chu kỳ message trở lên	0
Logic	ALL conditions (AND) hoặc ANY condition (OR)	ALL
Optional	Không đạt cũng không tính là lỗi	Tắt
Note	Ghi chú, sẽ hiện trong báo cáo	Trống

Signal mới thêm có Operator là ==. Value lấy theo thứ tự ưu tiên: giá trị tại cursor 1 nếu signal đang được vẽ và cursor đang bật, nếu không thì tên đầu tiên trong value table, cuối cùng là 0.

Xoá một step mà step khác đang lấy làm mốc After thì mốc của step kia tự về "Previous step".

Bước 3: Invariants và Abort

Hai nhóm này không bắt buộc. Chọn dòng nhóm trên cây rồi bấm **Signal** để thêm điều kiện, giống như với Trigger.

Nhóm	Khi nào dùng	Logic	Ví dụ
Invariants	Điều kiện phải đúng suốt từ Trigger tới khi kết thúc lần kiểm tra	Luôn ALL (AND), không đổi được	BMS_IsoFault == 0
Abort	Khi điều kiện đúng thì lần kiểm tra bị huỷ (ABORTED), không tính là FAIL	Mặc định ANY (OR)	IGN_Sts == OFF giữa chừng

Dùng Abort cho các tình huống người lái chủ động cắt ngang, để chúng không bị báo là lỗi của ECU.

Bước 4: Validate và lưu

Sequence được kiểm tra lại sau mỗi lần sửa: dòng lỗi có icon đỏ, dòng cảnh báo có icon vàng, dòng trạng thái dưới cây báo số lỗi và lỗi đầu tiên. Bấm **Validate** để xem danh sách đầy đủ; đóng hộp thoại thì cây tự nhảy tới dòng có vấn đề đầu tiên.

Mức	Thông báo thường gặp	Cách sửa
Lỗi	add at least one trigger condition	Thêm signal vào Trigger
Lỗi	add at least one step / the step has no condition	Thêm step hoặc thêm signal vào step
Lỗi	signal … not found in the loaded DBC files	Nạp đúng DBC, hoặc chọn lại signal
Lỗi	min … ms > max … ms	Sửa Window
Lỗi	the reference must be an earlier step	Đổi After sang step phía trước
Lỗi	bad value / 'between' needs two values	Sửa Value
Cảnh báo	found in network … instead of …	Kiểm tra tên network trong CAN Database
Cảnh báo	outside the DBC range / not in the value table	Kiểm tra lại Value

Mức	Thông báo thường gặp	Cách sửa
Cảnh báo	the step windows add up to ... ms, more than the timeout	Tăng Timeout hoặc thu hẹp Window
Cảnh báo	window ... ms is narrower than the ... ms cycle of ...	Mở rộng Window ít nhất bằng chu kỳ message được nêu
Cảnh báo	hold ... ms is shorter than the ... ms cycle of ...	Đặt Hold từ 2 chu kỳ trở lên, hoặc để 0
Cảnh báo	no timeout and step ... has no max	Đặt Timeout, hoặc max cho step đó

Signal chỉ cập nhật mỗi chu kỳ message, nên thời điểm đo được có thể trễ tới một chu kỳ. Hai cảnh báo theo chu kỳ lấy chu kỳ từ thuộc tính GenMsgCycleTime trong DBC; message không có thuộc tính này thì không được kiểm. Với cửa sổ, tool so với message chậm nhất của cả step lẫn mốc After.

Lưu bằng **Save** hoặc Ctrl+S. Lần đầu sẽ hỏi tên. File nằm ở `<thư mục tool>/Sequence Check/<tên>.json`, dùng chung cho mọi project. Muốn gửi cho người khác thì dùng **...** › **Export**, người nhận dùng **Import**.

Khi đổi sequence trong combo hoặc đóng Graphic mà còn thay đổi chưa lưu, tool hỏi Save / Discard / Cancel. Project nhớ sequence đang mở và mở lại nó lần sau.

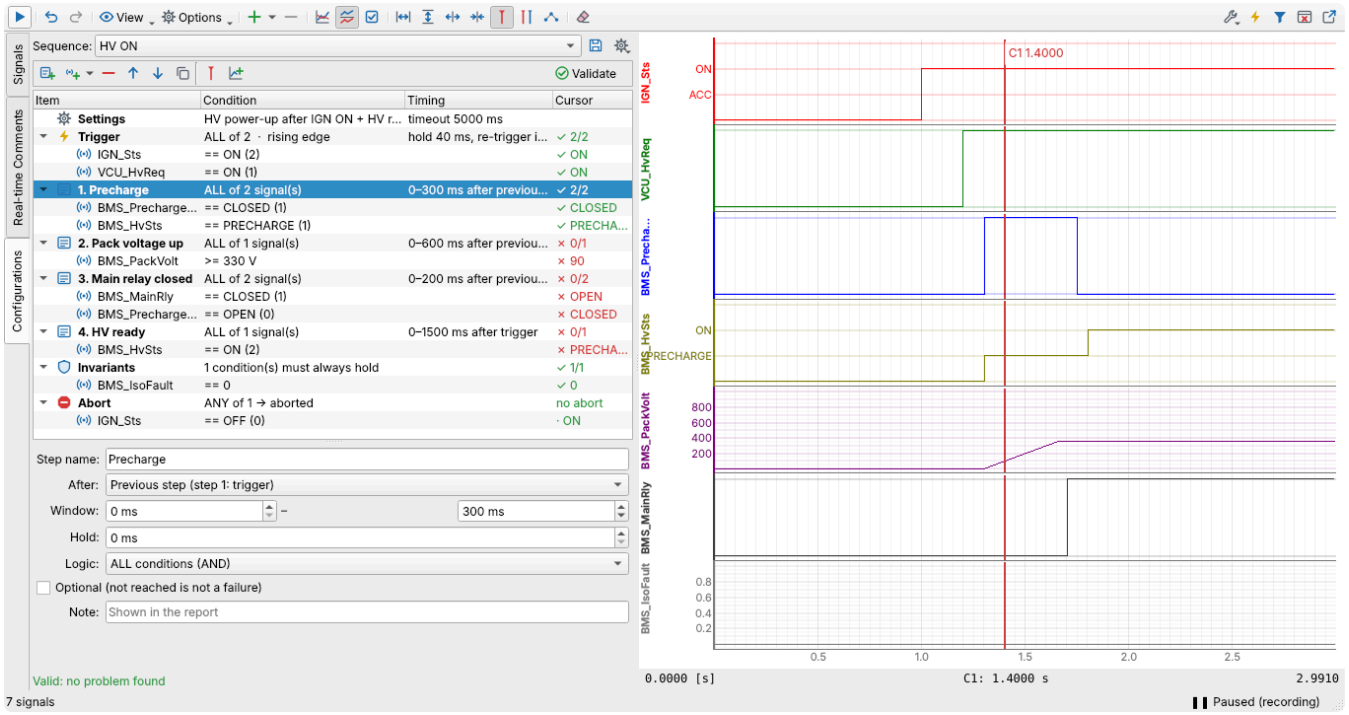
Bước 5: Đối chiếu với log

Phát log vào Graphic bằng Playback, rồi đặt cursor 1 tại từng thời điểm: cột **Cursor** trên cây cho biết ngay step nào đạt. Không cần phần cứng, không cần bấm HOME › Start.

1. Trong Sequence editor bấm **Plot** để đưa mọi signal của sequence sang tab Signals. Làm bước này **trước** khi phát log, vì Graphic chỉ giải mã các signal có trong tab Signals.
2. Mở **Bus › Bus Monitor**, tab **Playback**. Bấm Add files (hoặc kéo thả file log vào bảng).
3. Kiểm tra bảng mapping: mỗi kênh trong log phải trỏ tới đúng network trong CAN Database. Tool đã gợi ý sẵn, chỉ cần sửa nếu sai.
4. Chọn tốc độ (Fast để phát nhanh nhất) và khoảng thời gian nếu chỉ cần một đoạn, rồi bấm Start. Graphic tự chạy và dùng thời gian trong log; phát xong thì Graphic dừng và giữ dữ liệu.
5. Bật **Cursor 1** trên toolbar Graphic và click vào đồ thị tại thời điểm cần xem. Phím mũi tên trái/phải nhảy tới mẫu trước/sau của signal đang chọn, giúp đứng đúng chỗ chuyển trạng thái.
6. Đọc cột Cursor trên cây sequence (bảng dưới).

Cột Cursor hiện	Ý nghĩa
✓ CLOSED (xanh)	Điều kiện đúng tại cursor 1, kèm giá trị
✗ OPEN (đỏ)	Điều kiện sai tại cursor 1
✓ 2/2 hoặc ✗ 1/2	Ở dòng step hoặc nhóm: số điều kiện đúng trên tổng, theo Logic ALL/ANY
⚠ aborts / no abort	Ở dòng Abort: ngược lại, đúng nghĩa là sẽ huỷ nên hiện đỏ
not plotted	Signal chưa có trong tab Signals: bấm Plot
n.a.	Chưa có giá trị tại cursor 1 (trước mẫu đầu tiên)

Cột Cursor chỉ đọc cursor 1. Khi tắt cursor 1, cột **Value** của tab Signals chuyển sang hiện giá trị mới nhất (header có chấm tròn xanh) và cập nhật khoảng 5 lần mỗi giây trong lúc phát log hoặc đo thật; bật lại cursor 1 thì cột quay về giá trị tại cursor.



Tại 1,40 s: Trigger và step 1 đúng (✓ 2/2), PackVolt mới 90 V nên step 2 chưa đạt (✗ 0/1), step 3 và 4 chưa tới, Invariants đúng và chưa có Abort.

Tạo nhanh từ một log mẫu đúng

Với log có một lần chuyển trạng thái đúng, có thể lấy giá trị mong đợi thẳng từ log thay vì gõ tay:

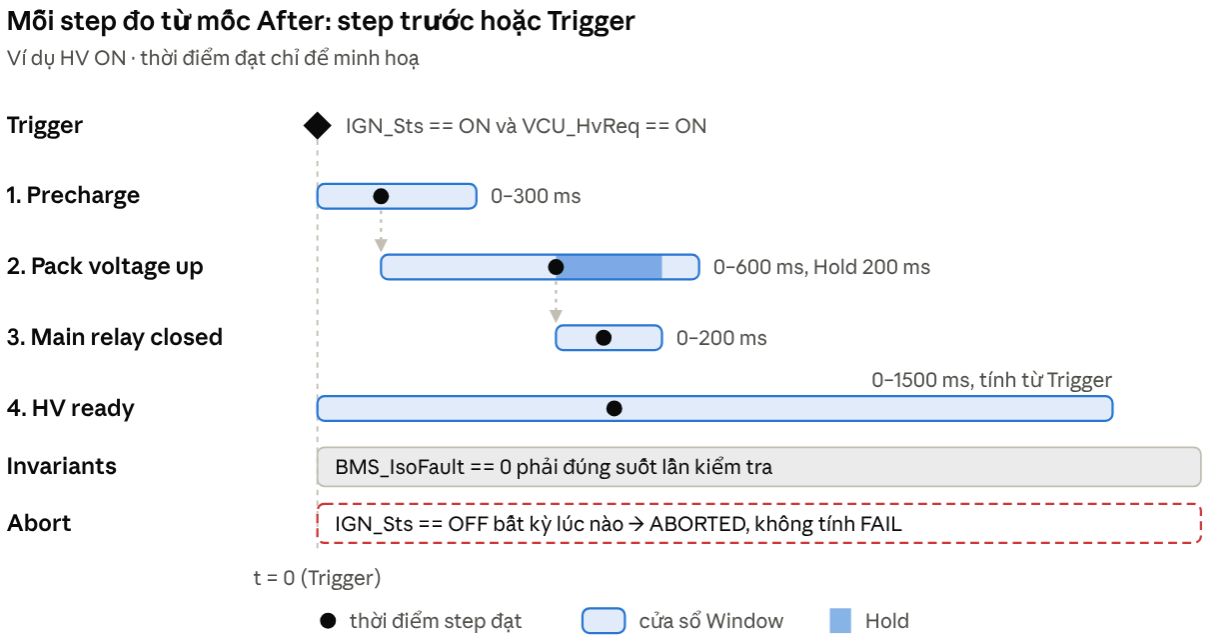
1. Đặt cursor 1 tại thời điểm step đã đạt trạng thái.

2. Chọn step (hoặc một điều kiện) và bấm **From cursor**. Value của các điều kiện được đặt bằng giá trị tại cursor; điều kiện between/outside giữ nguyên.
3. Để lấy số cho Window: bật thêm **Cursor 2**, đặt hai cursor tại mốc After và tại lúc step đạt. Graphic hiện Δt giữa hai cursor; cộng thêm biên độ cho phép rồi điền vào Window.

Ví dụ: chuỗi bật HV

Sequence "HV ON" kiểm tra BMS đóng relay đúng thứ tự và báo HV ON trong 1,5 s sau khi có IGN ON và yêu cầu HV. Settings: Timeout 5000 ms; Trigger dùng Logic ALL, Start "When it becomes true", Hold 40 ms, While running "Ignore new triggers". Giả sử VCU_Cmd gửi mỗi 20 ms và BMS_Status mỗi 100 ms, nên Hold của Trigger và step 2 đều bằng 2 chu kỳ.

Phần	Điều kiện	After	Window (ms)	Hold (ms)
Trigger	IGN_Sts == ON và VCU_HvReq == ON	–	–	–
1. Precharge	BMS_PrechargeRly == CLOSED và BMS_HvSts == PRECHARGE	Previous step	0 – 300	0
2. Pack voltage up	BMS_PackVolt >= 330	Previous step	0 – 600	200
3. Main relay closed	BMS_MainRly == CLOSED và BMS_PrechargeRly == OPEN	Previous step	0 – 200	0
4. HV ready	BMS_HvSts == ON	Trigger	0 – 1500	0
Invariants	BMS_IsoFault == 0	–	–	–
Abort	IGN_Sts == OFF	–	–	–



Sequence HV ON · mốc After và cửa sổ của 4 step

Mũi tên chấm: step 2 và 3 tính cửa sổ từ lúc step trước đạt; step 1 và step 4 tính từ Trigger.

Step 4 lấy mốc After là Trigger, nên nó giới hạn tổng thời gian cả chuỗi, bất kể các step giữa mất bao lâu. Tổng Window của step 1–3 là 1100 ms, nhỏ hơn Timeout nên Validate không cảnh báo.

Mẹo thiết kế sequence

Một sequence tốt chỉ kiểm một chức năng, bắt đầu từ sự kiện bên ngoài và có cửa sổ thời gian khớp với chu kỳ message.

- **Mỗi chức năng một sequence** (HV ON, HV OFF, Charging...). Đặt tên theo chức năng và ghi Description.
- **Trigger là sự kiện khởi đầu** do người lái hoặc ECU khác gây ra (IGN ON, yêu cầu HV), không phải trạng thái của chính ECU đang kiểm.
- **Thêm một step có mốc After là Trigger** để giới hạn tổng thời gian cả chuỗi, như step 4 trong ví dụ HV ON.
- **Window rộng hơn chu kỳ message chậm nhất; Hold khoảng 2 chu kỳ.** Hold của Trigger cũng vậy.
- **Signal analog:** dùng >=, <=, between hoặc Tolerance thay vì ==, vì giá trị vật lý hiếm khi trùng đúng một số.
- **Signal enum:** chọn tên trong value table (CLOSED, ON) thay vì gõ số để dễ đọc.
- **Step sau một step Optional** nên lấy mốc After là Trigger hoặc một step bắt buộc, vì step Optional có thể không xảy ra.

- **Abort cho thao tác cắt ngang** (tắt IGN giữa chừng), **Invariants cho lỗi không được xuất hiện** (lỗi cách điện).
- **Lấy số từ một log mẫu đúng:** From cursor cho giá trị, hai cursor và Δt cho thời gian, rồi cộng thêm biên.
- **Gom signal của sequence thành một group** trong tab Signals (Ctrl+G), đặt tên trùng sequence. Bấm vào dòng group thì mọi đường của group được tô đậm trên đồ thị.

Xử lý sự cố

Hiện tượng	Nguyên nhân	Cách xử lý
Bấm Signal thì báo "No DBC is loaded"	Chưa nạp DBC trong CAN Database	Nạp DBC cho network rồi thêm lại
Validate báo "not found in the loaded DBC files"	DBC khác bản, hoặc signal đã đổi tên	Nạp đúng DBC, hoặc xoá điều kiện và thêm lại signal mới
Cảnh báo "found in network X instead of Y"	Tên network trong CAN Database khác tên lúc tạo sequence	Đặt lại tên network cho trùng
Cột Cursor hiện "not plotted"	Signal chưa có trong tab Signals	Bấm Plot
Cột Cursor hiện "n.a."	Cursor 1 đang tắt, hoặc đặt trước mẫu đầu tiên	Bật Cursor 1 và đặt vào vùng có dữ liệu
Phát log xong mà đồ thị trống	Bấm Plot sau khi phát, hoặc kênh trong log trỏ sai network	Bấm Plot rồi phát lại; kiểm tra bảng mapping trong Playback
Không bao giờ thấy cảnh báo theo chu kỳ	DBC không khai báo GenMsgCycleTime	Chạy script ở mục Tự kiểm tra để xem; bổ sung thuộc tính này trong DBC
Cột Value đứng yên khi đang đo	Cursor 1 đang bật, hoặc Graphic đang Pause	Tắt Cursor 1, hoặc bấm Run
Đổi sequence trong combo thì bị hỏi Save / Discard / Cancel	Còn thay đổi chưa lưu	Chọn Save để giữ, Discard để bỏ

Tự kiểm tra công cụ bằng script

Script `check_sequence_rules.py` kiểm tra phần lưu/đọc sequence và các cảnh báo của Validate mà không cần giao diện hay phần cứng. Dùng nó sau mỗi lần cập nhật code, hoặc khi nghi DBC thiếu chu kỳ message.

1. Đặt file vào thư mục gốc project, cùng cấp với `core/` và `ui/`.
2. Trong PyCharm, click phải vào file › **Run 'check_sequence_rules'**, hoặc gõ trong Terminal (Alt+F12): `python check_sequence_rules.py`
3. Kết quả đúng: mọi dòng `PASS` và dòng cuối `ALL PASSED: 24/24 checks`. Có dòng `FAIL` thì gửi lại toàn bộ output.
4. Kiểm với DBC và sequence thật (gõ trên một dòng, hoặc dán phần từ `--dbc` vào Run › Edit Configurations › Parameters):

```
python check_sequence_rules.py --dbc "D:\DBC\Powertrain.dbc" --network
Powertrain --sequence "<thư mục tool>\Sequence Check\HV ON.json"
```

Output của bước 4 gồm ba phần:

- **Bảng chu kỳ message.** Dòng "0 co GenMsgCycleTime" nghĩa là DBC không khai báo chu kỳ, nên sẽ không có cảnh báo theo chu kỳ.
- **Mỗi step:** chu kỳ của step, của mốc After và độ phân giải (số lớn hơn). Window nên rộng hơn độ phân giải.
- **Danh sách Validate:** phải trùng với những gì editor báo trên cây.

Lỗi khi chạy script	Cách sửa
No module named 'core'	File chưa nằm ở thư mục gốc project
cannot import name 'RETRIGGER_IGNORE' / 'group_cycle'	Chưa chép model.py, validate.py bản mới vào core/sequence/
No module named 'cantools'	`pip install cantools` trong Terminal của PyCharm
FileNotFoundError	Đường dẫn --dbc hoặc --sequence sai; để đường dẫn trong dấu ngoặc kép

Tham khảo nhanh

Phím tắt và thao tác chuột

Phím / thao tác	Tác dụng
Ctrl+S	Lưu sequence đang mở
Del (trên cây sequence)	Xoá step hoặc điều kiện đang chọn
Click phải trên cây sequence	Menu thêm step, thêm signal
Ctrl+G (tab Signals)	Gom các signal đang chọn thành group
Click trái vào đồ thị	Đưa cursor gần nhất tới đó
← / → (đồ thị đang có focus)	Cursor 1 nhảy tới mẫu trước / sau của signal đang chọn
Shift + ← / →	Như trên, cho cursor 2

Định dạng file sequence

Mỗi sequence là một file JSON trong `<thư mục tool>/Sequence Check/<tên>.json`. Có thể mở bằng trình soạn thảo văn bản để xem hoặc so sánh, nhưng nên sửa trong editor để được Validate.

Trường	Ý nghĩa
format, version	Luôn là "MS_Tool.SequenceCheck", 1
name, description	Tên và mô tả
timeout_ms	Timeout; 0 = no limit
trigger.logic	"all" (AND) hoặc "any" (OR)
trigger.edge	"rising" hoặc "rising_or_initial" (đúng sẵn ở đầu log cũng tính)
trigger.hold_ms	Hold của Trigger
trigger.retrigger	"ignore", "restart" hoặc "parallel"
steps[].reference	"previous", "trigger" hoặc id của một step phía trước
steps[].min_ms, max_ms, hold_ms	Window và Hold; max_ms = 0 là no limit

Trường	Ý nghĩa
steps[].optional, note	Optional và ghi chú
invariants, abort	Nhóm điều kiện: logic + conditions
conditions[]	network, message, id ("0x100"), signal, op, value, value2 (between/outside), tolerance

```
{
  "format": "MS_Tool.SequenceCheck", "version": 1,
  "name": "HV ON", "timeout_ms": 5000,
  "trigger": {"logic": "all", "edge": "rising", "hold_ms": 40, "retrigger":
"ignore",
              "conditions": [{"network": "Powertrain", "message": "VCU_Cmd",
"id": "0x200",
                              "signal": "IGN_Sts", "op": "==", "value":
"ON"}]},
  "steps": [{"id": "62bc33c0", "name": "Precharge", "reference": "previous",
            "min_ms": 0, "max_ms": 300, "hold_ms": 0, "optional": false,
"logic": "all",
            "conditions": [{"network": "Powertrain", "message":
"BMS_Status", "id": "0x100",
                            "signal": "BMS_PrechargeRly", "op": "==",
"value": "CLOSED"}]}],
  "invariants": {"logic": "all", "conditions": []},
  "abort": {"logic": "any", "conditions": []}
}
```

Giới hạn hiện tại

Bản hiện tại mới soạn, validate và đối chiếu thủ công tại cursor 1; chưa tự chạy sequence trên toàn bộ log để ra PASS / FAIL / ABORTED.

- Trigger (kể cả Hold và While running), Window, Hold, Invariants và Abort đã được lưu và kiểm tra cú pháp, nhưng chưa được tính tự động theo thời gian.
- Cột Cursor chỉ dùng cursor 1 và chỉ với signal có trong tab Signals.
- Phần sẽ làm tiếp: chạy sequence như một Post Process trong Playback, cửa sổ kết quả riêng, báo cáo Excel và marker kết quả trên Graphic.